ARCHITECTURE REFERENCE • DRAFT FOR IMPLEMENTATION

Fabric Try-on AI प्रस्तावित Architecture

प्रत्येक garment साठी स्वतंत्र prompt, poses आणि generation defaults ठेवून AI code कसा विभागायचा याचा संदर्भ दस्तऐवज.

उद्देश: पुढे वेगवेगळ्या फाईल्स स्वतंत्रपणे तयार करताना हा दस्तऐवज ChatGPT ला संदर्भ म्हणून द्यायचा. हा architecture plan आहे; या दस्तऐवजामुळे code बदललेला नाही.

मराठी संदर्भ • Version 1 • 22 ऑक्टोबर 2026

1. या architecture ची मान्य केलेली दिशा

- AI flow साठी `fabricapp/ai/main.py` ठेवायची. Django चा विद्यमान `manage.py` त्याच्या जागी येत नाही.
- प्रत्येक garment type चा स्वतःचा package आणि स्वतंत्र `builder.py`, `prompts.py`, `poses.py` आणि `settings.py` असेल.
- `seed` आणि `guidance` हे दोन generation parameters प्रत्येक garment साठी स्वतंत्र defaults असतील. एका garment चे बदल दुसऱ्याला लागू होणार नाहीत.
- Image ordering/roles साठी स्वतंत्र `image_plan.py` नको. सध्याची common image योजना shared engine flow मध्ये राहील.
- Background, image processing, model API call आणि generic lookup यांसारखी खरोखर common कामे common modules मध्ये राहतील.
- Saree चे सध्याचे स्वतंत्र package ही पहिली उदाहरण रचना आहे. Saree चे prompt, poses आणि pallu नियम इतर garment packages मध्ये मिसळायचे नाहीत.
- सध्याचे prompt शब्द आणि output behavior जसे आहेत तसे ठेवायचे. Refactor म्हणजे code योग्य फाईलमध्ये हलवणे/connection बदलणे; prompt rewrite किंवा tuning नाही.

> **Default values:** project code मध्ये `guidance = 7.0` आणि `seed = 42` अनेक ठिकाणी आढळले. वापरकर्त्याला पूर्वी आठवलेली `24` ही seed/value पाहिलेल्या संबंधित files मध्ये सापडली नाही. Migration करताना प्रत्येक route ची सध्याची प्रत्यक्ष value तपासून तशीच संबंधित garment default मध्ये हलवावी; 24 गृहित धरू नये.

2. पूर्ण प्रस्तावित project tree

खाली implemented garment types साठी पूर्ण target tree दाखवला आहे. नवा garment जोडल्यावर त्याच package pattern ने folder वाढेल. सध्या unimplemented प्रकारासाठी रिकामी package तयार करण्याची गरज नाही.

```
fabric_tryon/
├── manage.py                          # Django command-line entry point (विद्यमान)
├── tryon/
│   ├── __init__.py
│   ├── settings.py                    # Django configuration
│   ├── urls.py                        # project URL routing
│   ├── asgi.py
│   └── wsgi.py
├── fabricapp/
│   ├── __init__.py
│   ├── apps.py
│   ├── admin.py
│   ├── models.py
│   ├── serializers.py                 # API input validation
│   ├── urls.py                        # API routes
│   ├── views.py                       # request -> AI entry -> response
│   ├── tests.py
│   ├── services/
│   │   ├── __init__.py
│   │   └── cloudinary_service.py      # upload/store result URL
│   └── ai/
│       ├── __init__.py
│       ├── main.py                    # AI orchestration entry point
│       ├── router.py                  # scenario + garment dispatch
│       ├── cloudflare_engine.py       # shared model API/image flow
│       ├── image_utils.py             # shared resize/dimensions/canvas
│       ├── backgrounds.py             # common background options/text
│       ├── camera_data.py             # shared camera/framing primitives
│       ├── faces.py                   # common face choices/path lookup
│       ├── pose_registry.py           # common lookup of valid garment poses
│       ├── pose_reference.py          # common reference-image path lookup
│       ├── detail_engine.py           # generic garment-detail application
│       ├── garment_image.py           # common uploaded-garment prompt route
│       ├── debug_utils.py             # optional local debug file saving
│       ├── faces/
│       │   ├── anaya.png
│       │   ├── poonam.png
│       │   ├── raghav.png
│       │   └── vihaan.png
│       ├── poses/
│       │   ├── saree/                 # saree pose-reference photos
│       │   ├── kurta/
│       │   ├── kurti_pant/
│       │   ├── shirt/
│       │   ├── blazer/
│       │   └── pant/
│       ├── saree/
│       │   ├── __init__.py
│       │   ├── builder.py
│       │   ├── prompts.py
│       │   ├── poses.py
│       │   └── settings.py
│       ├── kurta/
│       │   ├── __init__.py
│       │   ├── builder.py
│       │   ├── prompts.py
│       │   ├── poses.py
│       │   ├── settings.py
│       │   └── details.py
│       ├── kurti_pant/
│       │   ├── __init__.py
│       │   ├── builder.py
│       │   ├── prompts.py
│       │   ├── poses.py
│       │   ├── settings.py
│       │   └── details.py
│       ├── shirt/
│       │   ├── __init__.py
│       │   ├── builder.py
│       │   ├── prompts.py
│       │   ├── poses.py
```

```
│       │   ├── settings.py
│       │   └── details.py
│       ├── blazer/
│       │   ├── __init__.py
│       │   ├── builder.py
│       │   ├── prompts.py
│       │   ├── poses.py
│       │   └── settings.py
│       └── pant/
│           ├── __init__.py
│           ├── builder.py
│           ├── prompts.py
│           ├── poses.py
│           └── settings.py
├── check_pose_prompts.py              # offline prompt preview/check
├── check_saree_prompts.py             # saree prompt preview/check
├── check_saree_regressions.py         # saree regression checks
├── verify_kurti_fixes.py              # existing kurti-pant checks
├── requirements.txt
├── Dockerfile
├── render.yaml
└── env-vars.yaml
```

> **Details file:** `details.py` फक्त ज्या garment कडे प्रत्यक्ष sleeve/tuck/dupatta सारखे API detail options आहेत त्यांच्यासाठी. पुढे इतर garment ला असे options जोडले तरच त्याच्या folder मध्ये ही फाईल जोडायची.

3. Request ते generated image: connection flow

Client → fabricapp/views.py → serializers.py → ai/main.py → ai/router.py → garment/builder.py → cloudflare_engine.py → model → Cloudinary service → API response

1. **`views.py`** request घेते आणि serializer कडून validated data मिळवते.
2. **`serializers.py`** input fields, garment type/style, pose आणि seed/guidance override वैध आहेत का तपासते.
3. **`ai/main.py`** validated request चा AI flow सुरू करते; scenario ठरवणे/dispatch करणे router कडे सोपवते.
4. **`ai/router.py`** garment type आणि scenario नुसार संबंधित builder निवडते. Garment prompt स्वतः इथे लिहायचा नाही.
5. **`<garment>/builder.py`** त्या garment चे prompt text, pose data, background/camera helper आणि settings एकत्र करून generation config देते.
6. **`cloudflare_engine.py`** image 0/1/2 ची सध्याची common भूमिका आणि order कायम ठेवून prompt, seed आणि guidance model ला पाठवते.
7. Output image परत आल्यानंतर विद्यमान service/storage path वापरून Cloudinary upload व API response पूर्ण होतो.

Image plan: कोणत्या scenario मध्ये blank canvas/person/fabric/reference कोणत्या image index ला जातात हा common flow आहे. या architecture मध्ये स्वतंत्र `image_plan.py` नाही. एखाद्या garment साठी भविष्यात खरा अपवाद आढळल्यास तेव्हा design पुन्हा ठरवायचे.

4. Common फाईल्स: प्रत्येकाची नेमकी जबाबदारी

फाईल	काय ठेवायचे	काय ठेवायचे नाही
`ai/main.py`	AI feature चा स्थिर entry point; request context घेऊन router/engine शी जोडणे.	Garment-specific prompt wording, poses, model API details.
`ai/router.py`	Garment type आणि scenario वरून योग्य package builder निवडणे; unsupported combination हाताळणे.	मोठे prompt strings, pose descriptions, per-garment defaults.
`cloudflare_engine.py`	Shared API request, model payload, image preparation/orchestration, response/error handling.	Garment prompt बनवणे किंवा saree/kurta चे नियम.
`image_utils.py`	Resize, aspect ratio/dimensions, blank canvas, common image buffers.	Garment semantics किंवा prompts.
`backgrounds.py`	सर्व garments मध्ये वापरता येतील असे background keys आणि base descriptions.	एखाद्या garment चा pose-specific prop/ अपवाद. तो garment builder मध्ये लागू होतो.
`camera_data.py`	Front/side/close-up सारखी shared framing selection व generic values.	“pant legs” सारखी विशिष्ट garment गृहितकं; गरज असल्यास builder कडून garment wording द्यावे.
`faces.py + faces/`	Face names, gender mapping आणि file path lookup; face images एकदाच साठवणे.	Garment-specific चेहरा किंवा garment prompt rules.
`pose_registry.py`	प्रत्येक package च्या `poses.py` मधून valid pose keys एकत्र शोधणे; serializer/API ला योग्य choices देणे.	Pose चे पूर्ण वाक्य/शरीरस्थितीचा मजकूर; तो प्रत्येक garment च्या `poses.py` मध्ये.
`pose_reference.py`	दिलेल्या garment+pose साठी reference photo path lookup चा generic helper.	सगळ्या garments ची reference यादी एकाच ठिकाणी. त्या नोंदी संबंधित garment च्या `poses.py` मध्ये.
`detail_engine.py`	सामान्य पद्धतीने निवडलेला detail prompt मध्ये लागू करणे.	Kurta/kurti/shirt चे विशिष्ट detail पर्याय व wording; ते त्या garment च्या `details.py` मध्ये.
`garment_image.py`	Fabric swatch ऐवजी तयार garment image दिल्यावरचा common prompt route/config.	Fabric-swatch garment-specific prompt.
`debug_utils.py`	ऐच्छिक local debug copies/logging; production setting बंद असताना no-op.	Generation decisions किंवा garment behavior.

5. प्रत्येक garment package मधील फाईल्स

फाईल	जबाबदारी	उदाहरणार्थ काय बदलेल
`__init__.py`	Package मधील सार्वजनिक builder function export करणे.	Router ला package कडून builder import करता यावा.
`builder.py`	Scenario resolve करणे; लागू prompt sections निवडणे; common helpers जोडणे; seed/guidance config परत करणे.	त्या garment च्या own-model, person-photo, pose-reference मार्गांची रचना.
`prompts.py`	त्या garment चे prompt text, section labels, garment/fabric/modesty-specific wording.	Prompt wording बदलायचे असल्यास फक्त मंजुरीनंतर आणि वेगळ्या कामात. या migration मध्ये शब्द बदलायचे नाहीत.
`poses.py`	त्या garment चे pose keys व descriptions; saree मध्ये pallu placement/pose props; त्या garment साठी pose-reference image metadata.	नवीन pose जोडणे किंवा pose text दुरुस्त करणे; इतर garments च्या poses वर परिणाम होऊ नये.
`settings.py`	त्या garment साठी स्वतंत्र default seed आणि guidance; प्रथम स्तरावर garment-level config.	Testing साठी त्या garment चा default tune करणे.
`details.py` (गरज असल्यास)	फक्त त्या garment चे detail slots/options, wording आणि style restrictions.	Folded sleeves, tucked/untucked, dupatta सारखे पर्याय; दुसऱ्या garment चे नियम इथे नाहीत.

Subtypes: या पहिल्या स्तरात blazer/kurta सारख्या subtype ची settings सुद्धा त्याच garment च्या `settings.py` मध्ये keyed mapping म्हणून राहू शकतात. पुढे प्रत्येक subtype+parameter combination ला स्वतंत्र config लागला तर त्यावेळी विस्तार करायचा; आत्ता स्वतंत्र subtype folders/files ची गरज नाही.

6. दोन generation parameters: seed आणि guidance

प्रत्येक garment चा default

```
# saree/settings.py
DEFAULT_SEED = <current saree route value>
DEFAULT_GUIDANCE = <current saree route value>

# kurta/settings.py
DEFAULT_SEED = <current kurta route value>
DEFAULT_GUIDANCE = <current kurta route value>
```

- Default value per-garment आहे; `common/settings.py` मधून सगळ्यांना एकच value देऊ नये.
- API request मध्ये `seed` किंवा `guidance` न आल्यास निवडलेल्या garment ची default वापरायची.
- Testing override आल्यास फक्त त्या request साठी override वापरायचा; इतर garment चा default बदलत नाही.
- Override ला numeric type/range validation ठेवावी. Engine ला पाठवताना model API चा सध्याचा field name आणि type कायम ठेवावा.
- Subtypes साठी पहिल्या स्तरावर एकच garment default लागू; पुढे subtype-specific values add करता येतील.

प्रस्तावित parameter flow

```
serializer validated data:
  seed?       -> optional integer override
  guidance?   -> optional numeric override

router selects garment builder
builder selects its settings.py defaults
request override, if present, replaces only that request's default
cloudflare_engine sends resolved seed + guidance to the model
```

> **महत्त्वाचे:** आधीचे configured values प्रत्येक route मधून जसेच्या तसे उचलायचे. Code मध्ये काही जागी `42/7.0` दिसतात, पण प्रत्येक scenario/subtype साठी एकसारखे आहेत असे migration करण्यापूर्वी verify करायचे. `24` आढळल्यास त्याच संबंधित garment/scenario च्या default मध्ये ठेवायचे. या PDF मध्ये कोणताही नवीन tuning result ठरवलेला नाही.

Seed/guidance बदल output वर परिणाम करू शकतात; पण background मुळे face quality कमी होण्याचे कारण हेच आहे असे आधीच ठरवलेले नाही. Parameters स्वतंत्रपणे बदलून तुलना करण्यासाठी optional request override उपयोगी आहे.

7. सध्याच्या code मधून काय कुठे हलवायचे

सध्याचा स्रोत	प्रस्तावित target	Refactor करताना नियम
`fabricapp/ai/prompt_builder.py`	Common dispatch → `router.py`; saree prompt composition → `saree/builder.py`; बाकी garment composition संबंधित builders मध्ये.	Scenario order/behavior preserve करा; prompt strings edit किंवा simplify करू नका.
`fabricapp/ai/prompts.py`	Garment prompt data संबंधित `<garment>/prompts.py`; background data → `backgrounds.py`; face data → `faces.py`; garment-image prompt → `garment_image.py`.	Prompt चे शब्द byte-for-byte/अर्थाने जसाच्या तसा हलवावेत; common data ची duplicate copy नको.
`fabricapp/ai/pose_data.py`	Pose wording/overrides प्रत्येक `<garment>/poses.py`; available-key aggregation → `pose_registry.py`.	प्रत्येक garment ला त्याचे valid posesच दिसावेत; API choices आणि compatibility जपावी.
`fabricapp/ai/prompt_parts.py`	Saree-specific prompt text → `saree/prompts.py`.	हे common नाही. Saree resolver आणि pose-reference prompt दोन्ही मजकूर हरवू नयेत.
`fabricapp/ai/saree/`	Target `fabricapp/ai/saree/` package म्हणून कायम.	सध्याचे `builder.py/poses.py/texts.py` नीट पाहून prompt data एकत्र कुठे ठेवायचा ते ठरवा; मजकूर किंवा output behavior बदलू नका.
`fabricapp/ai/camera_data.py`	`camera_data.py` common; garment-sensitive framing wording संबंधित builder मध्ये.	Generic close-up मध्ये pant/saree सारख्या चुकीच्या गृहितके ठेवू नयेत.
`fabricapp/ai/garment_details.py`	Generic replacement logic → `detail_engine.py`; per-garment rules → package-local `details.py`.	Exact prompt anchors बदलू नयेत; options API compatibility जपावी.
`fabricapp/ai/cloudflare_engine.py` आणि `image_utils.py`	Shared AI engine आणि image utilities म्हणून राहतील.	फक्त builder/config connection बदलायचा; image ordering, dimensions, upload behavior अनावश्यकपणे बदलू नये.
`serializers.py`, `views.py`	नवीन seed/guidance optional inputs validate/forward; AI flow `main.py` ला call.	जुने input/output contract जपावे; prompt निर्माण इथे करू नये.

8. Refactor करताना पाळायचे acceptance rules

- या कामाचा उद्देश architecture split आणि wiring आहे; prompt wording सुधारणा नाही.
- जुन्या prompt text च्या versions/order/closing rules जतन करा. काही string हलवावी लागली तर तिचा मजकूर बदलू नका.

- प्रत्येक garment च्या pose list, pose text, seed आणि guidance स्वतंत्र आहेत. Common registry/engine हा lookup/transport करतो; values स्वतःची मालकी common कडे घेत नाही.
- Image plan वेगळी file नाही. Existing image-role order सर्व scenarios मध्ये जसाच्या तसा ठेवायचा.
- Pose-reference images garment folders मध्ये ठेवा; generic path lookup common राहू शकतो.
- एक garment बदलून इतर garment च्या prompt/config वर परिणाम होऊ नये.
- API मधून optional overrides नसतील तर जुने per-route valuesच वापरले जातील.
- Refactor नंतर import paths, serializer pose choices, style options, details validation, image reference paths आणि model payload wiring तपासा.

9. स्वतंत्रपणे code generate करण्याचा सुरक्षित क्रम

1. हा PDF ChatGPT conversation मध्ये attach/upload करा. सुरुवातीला "हा architecture reference आहे; आधी फक्त एक फाईल तयार कर" अशी सूचना द्या.
2. प्रथम public data contracts ठरवा: builder input/output shape, scenario keys, prompt config keys (`prompt`, `seed`, `guidance`)—existing values/text untouched.
3. Common foundations: `backgrounds.py`, `faces.py`, `camera_data.py`, `pose_registry.py`, `pose_reference.py`, `image_utils.py`, `detail_engine.py`.
4. सर्वात आधी saree package migrate करून prompt text आणि behavior जसेच्या तसे राहिले आहेत याची स्वतंत्र खात्री करा.
5. त्यानंतर एकावेळी एक garment package migrate करा; प्रत्येक `settings.py` मध्ये त्या garment चे सध्याचे defaults आले आहेत हे नोंदवा.
6. मग `router.py`, `main.py`, `cloudflare_engine.py` आणि views/serializer यांची जोडणी करा.
7. प्रत्येक file वेगळी generate करताना संबंधित source file upload/paste करा. Prompt strings बदलू नका; जबाबदाऱ्या वेगळ्या करून public function compatibility जपा, अशी सूचना द्या.

> **Code generation साठी लक्षात ठेवा:** ChatGPT कडे संपूर्ण codebase context नसेल, म्हणून प्रत्येक file तयार करताना संबंधित विद्यमान source file(s), या PDF मधील target role, imports/callers आणि अपेक्षित function signature सोबत द्या. आधी interface मागा; मग implementation. त्यामुळे वेगळ्या sessions मध्ये बनलेल्या files चे connections जुळवणे सोपे होईल.

10. ChatGPT कडे context नेण्याबद्दल

या Codex chat ची internal memory किंवा thread state सामान्य ChatGPT chat मध्ये थेट पाठवण्याची सुविधा माझ्याकडे नाही. हा PDF मात्र architecture context म्हणून upload करता येईल. त्यामुळे पूर्ण chat transfer न करताही project direction, file responsibilities आणि refactor constraints ChatGPT ला समजतील.

PDF upload केल्यानंतर हा छोटा context message paste करा:

```
मी Fabric Try-on Django project साठी AI architecture refactor करत आहे. Attached PDF हा मंजूर architecture reference आहे. मी files वेगवेगळ्या sessions मध्ये तयार करेन. आधी दिलेली file आणि तिचे callers पाहा. सध्याचे prompt text, scenario behavior, image order आणि API contract अनावश्यकपणे बदलू नका. प्रत्येक garment ला स्वतःचे builder.py, prompts.py, poses.py, settings.py द्या; seed आणि guidance defaults garment-specific ठेवा. Request मधील optional seed/guidance override फक्त त्या request साठी वापरा. Common files मध्ये फक्त खरोखर shared mechanisms ठेवा. प्रत्येक वेळी एकच file generate करा आणि त्याचे imports/expected interface स्पष्ट लिहा.
```

तयार केलेला दस्तऐवज हा architecture reference आहे. या कामात prompt text किंवा application code बदललेले नाही.